QUICK REFERENCE 3-PAGER • A4

Visualization Tool

Turn any dataset + question into a chart in one call — DuckDB aggregates, Vega-Lite spec out

DeepAgent tool **create_visualization** · **create_visualizations** (HTML/Plotly variant) · Deterministic, no LLM hallucination on numbers

WHAT IT IS One tool call: datasets + question → chart_spec + facts + warnings

■ Deterministic
DuckDB computes every aggregate over full data. No sampling for final values. Numbers are facts, not LLM guesses.

■ Declarative
Output is a **Vega-Lite JSON spec** — portable, themeable, renders anywhere. No binary image blobs.

■ Auto chart type
auto infers bar/line/pie/scatter from the question. Override with explicit **chart_type** when needed.

INPUTS & OUTPUTS

Pydantic v2 strict schemas • extra="forbid"

■ INPUT VisualizationRequest

Field	Type / Values	Notes
datasets	list[**DataRef**] (min 1, unique names)	Inline / URL / connector
DataRef.kind	**inline_json • inline_csv • url • connector**	kind dictates required field
DataRef.name	str 1–128 chars, unique per request	Used as DuckDB table name
DataRef.data	Any (for inline kinds)	JSON array or CSV string
DataRef.url	str (for url kind)	Fetched via httpx + pagination
question	str (min 1 char, required)	NL description of desired chart
chart_type	**auto** • bar • line • pie • scatter • table	Default **auto** — keyword inference
options	**VisualizationOptions** (optional)	See options table below

■ OUTPUT VisualizationResponse (JSON string from tool)

Field	Type	Description
chart_spec	dict (Vega-Lite v5)	Declarative spec: $schema, data, mark, encoding
facts	list[dict]	Computed aggregates (group_sum, count, etc.)
warnings	list[str]	Non-fatal: truncation, empty, agg fallback
meta	dict	chart_type, x_field, y_field, row_count
error	dict (on failure)	error_code, message, details.exception

VisualizationOptions All optional • strict, extra="forbid"

Option	Type / Default	Constraints	Effect
max_rows	int | None	ge=1	Cap rows per dataset; warns on truncation
width	int default 400	100–2000 px	Vega-Lite width
height	int default 300	100–2000 px	Vega-Lite height
locale	str default "en-US"	BCP-47	Number/date formatting
include_warnings	bool default true	—	Include warnings in response

DataRef Kinds How datasets are supplied

inline_json	data: **list[dict]**	JSON array of rows	✔ inline
inline_csv	data: **str** (CSV)	CSV string + options.delimiter	✔ inline
url	url: **str**	Fetched via httpx, pagination	remote
connector	connector_config: **dict**	Connector params (extensible)	extensible

*All DataRef kinds share **name** (unique) + **options** (dict, e.g. delimiter/encoding). Validation: inline kinds require **data**, url requires **url**, connector requires **connector_config**.*

ARCHITECTURE & PIPELINE

From raw datasets to Vega-Lite spec

Architecture Diagram

Data ingestion → DuckDB aggregation → Chart selection → Vega-Lite spec

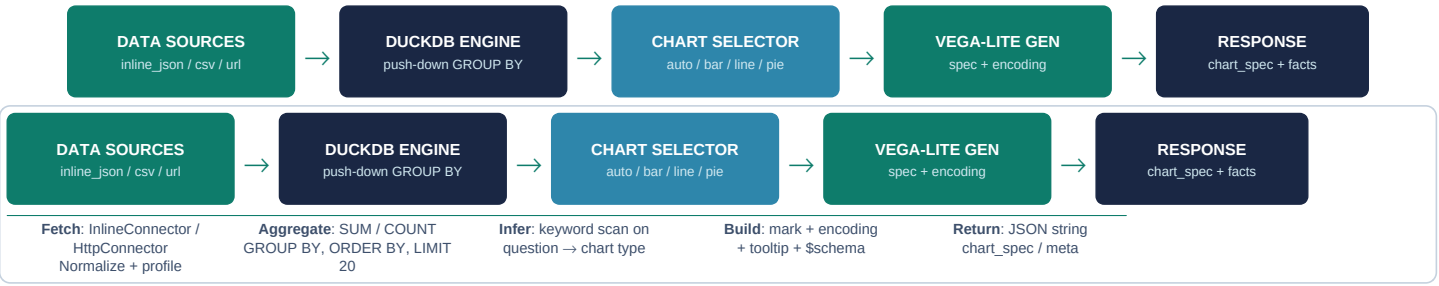


Fig. 1 — Architecture: datasets flow through DuckDB push-down aggregation, chart-type selection, and Vega-Lite spec generation. No LLM in the hot path for numbers.

Pipeline Flowchart

7 stages from fetch to response

- **Fetch** — Parse datasets_json; InlineConnector / HttpConnector per DataRef.kind; collect warnings for unknown kinds.
- **Normalize** — normalize_rows(): type coercion, null handling, date parsing; truncate to max_rows with warning.
- **Profile** — profile_dataset(): column stats, inferred types (string/integer/float), cardinality, 5-row sample.
- **Infer** — _infer_chart_type(): keyword scan on question; explicit chart_type bypasses inference.
- **Aggregate** — DuckDBEngine GROUP BY: SUM(y) per x (bar/pie), raw rows (scatter), COUNT(*) fallback; LIMIT 20.
- **Build spec** — _build_vega_spec(): $schema v5, width/height, data.values, mark + encoding + tooltip per chart type.
- **Return** — JSON string: {chart_spec, facts, warnings, meta{chart_type, x_field, y_field, row_count}} or error envelope.
- **DuckDB push-down** — All aggregates run inside DuckDB; no Python loops over full data.

Fig. 2 — Pipeline: 7 stages. DuckDB handles aggregation; Vega-Lite spec is the portable output. Warnings accumulate at each stage.

Chart-Type Decision Matrix

How auto maps question keywords → chart type

Chart	Keywords in question	Mark	Encoding	When to use
bar (default)	no keyword match	bar	x: nominal, y: quantitative	Categorical comparison
line	trend, over time, time series, evolution	line + point	x: nominal, y: quantitative	Trends over time
pie	distribution, share, proportion, pie	arc	theta: quant, color: nominal	Part-to-whole share
scatter	scatter, correlation, relationship	point	x: quant, y: quant	Correlation / relationship
table	explicit chart_type only	—	—	Raw data view

*Explicit **chart_type** always wins. Auto fallback is **bar**. Scatter requires ≥2 numeric columns; otherwise falls back to COUNT(*) by string column.*

Chart-Type Selection Logic

- **_infer_chart_type(question, chart_type)** — if chart_type != "auto", return as-is.
- Otherwise lowercase question and scan keywords:

line: "trend" | "over time" | "time series" | "evolution"
pie: "distribution" | "share" | "proportion" | "pie"
scatter: "scatter" | "correlation" | "relationship"
bar: default fallback

- Field inference: first **string** col → x_field, first **numeric** col → y_field.
- Scatter: needs ≥2 numeric cols → x=numeric[0], y=numeric[1]; else COUNT(*) fallback.
- Aggregation: **bar/pie** → SUM(y) GROUP BY x; **scatter** → raw rows (limit 100); else COUNT(*) GROUP BY x.

Security & Sanitization

- **Strict Pydantic**: extra="forbid", unique dataset names, ge/le on width/height/max_rows.
- **Identifier sanitization**: re.sub(r"[^A-Za-z0-9_]", "_", table_name) before DuckDB.
- **No raw HTML**: output is JSON spec, not HTML — no XSS surface. Vega-Lite renders client-side.
- **Graceful degradation**: unknown kind → warning + skip; empty data → warning; agg failure → fallback rows.

Vega-Lite Spec Structure

Every chart_spec shares this skeleton; only **mark** + **encoding** vary:

Vega-Lite spec skeleton (bar example) — mark + encoding vary per chart type:

```
{
  "$schema": "https://vega.github.io/schema/vega-lite/v5.json",
  "width": 400,
  "height": 300,
  "data": {"values": [...]},
  "mark": "bar",
  "encoding": {
    "x": {"field": "category", "type": "nominal",
          "axis": {"labelAngle": -30}},
    "y": {"field": "value", "type": "quantitative"},
    "tooltip": [{"field": "category"},
```

```
                {"field": "value", "type": "quantitative"}]
  }
}
```

Chart	mark	encoding
bar	bar	x: nominal (labelAngle -30), y: quantitative + tooltip
line	{"type":"line","point":true}	x: nominal, y: quantitative + tooltip
pie	arc	theta: quantitative, color: nominal + tooltip
scatter	point	x: quantitative, y: quantitative + tooltip

API & DeepAgent Tool Usage

```
from visualization_tool.tool import create_visualization
import json

datasets = [{"kind": "inline_json", "name": "sales",
             "data": [{"region": "EMEA", "revenue": 42000},
                      {"region": "APAC", "revenue": 38000}]}]

result_json = create_visualization(
    datasets_json=json.dumps(datasets),
    question="Show revenue by region as a bar chart",
    chart_type="auto",          # or "bar" / "line" / "pie" / "scatter" / "table"
    options_json=json.dumps({"width": 500, "height": 300, "max_rows": 1000})
)
result = json.loads(result_json)
# result["chart_spec"]  -> Vega-Lite spec dict
# result["facts"]       -> [{"id":"agg_rows","kind":"group_sum","value":2,...}]
# result["warnings"]    -> []
# result["meta"]        -> {"chart_type":"bar","x_field":"region","y_field":"revenue","row_count":2}
```

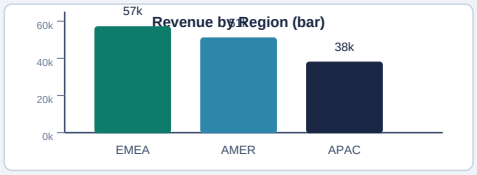
Tool	Function	Returns
create_visualization	create_visualization(datasets_json, question, chart_type="auto", options_json="{}")	JSON string
create_visualizations	HTML/Plotly variant (spec sibling) — returns self-contained HTML	text/html

*DeepAgent registration: **visualization_tool = create_visualization** + **VISUALIZATION_TOOL_SPEC** dict. Falls back to Pydantic create_model if langchain_core not installed. Async fetch via asyncio.run().*

Worked Mini-Example Revenue by region → bar chart

```
REQUEST
datasets_json: [{
  "kind": "inline_json",
  "name": "sales",
  "data": [
    {"region": "EMEA", "revenue": 42000},
    {"region": "APAC", "revenue": 38000},
    {"region": "AMER", "revenue": 51000},
    {"region": "EMEA", "revenue": 15000}
  ]
}]
question: "Bar chart of revenue by region"
chart_type: "auto"  -> resolves to "bar"
options: {"width": 400, "height": 300}
```

```
RESPONSE (json.loads)
{
  "chart_spec": {
    "$schema": "https://vega.github.io/.../v5.json",
    "width": 400, "height": 300,
    "data": {"values": [
      {"region": "EMEA", "revenue": 57000},
      {"region": "AMER", "revenue": 51000},
      {"region": "APAC", "revenue": 38000}
    ]},
    "mark": "bar",
    "encoding": {
      "x": {"field": "region", "type": "nominal",
            "axis": {"labelAngle": -30}},
      "y": {"field": "revenue", "type": "quantitative"}
    }
  },
  "facts": [{"id":"agg_rows","kind":"group_sum",
             "value":3,"formatted":"3"}],
  "warnings": [],
  "meta": {"chart_type":"bar","x_field":"region",
           "y_field":"revenue","row_count":3}
}
```



region	revenue
EMEA	57,000
AMER	51,000
APAC	38,000

Fig. 3 — Worked example: 4 rows grouped by region via DuckDB SUM(revenue) GROUP BY region → 3 aggregated rows → bar chart. Vega-Lite spec renders identically in any Vega-Lite runtime.

Key Files & Error Handling

- **src/visualization_tool/tool.py** — create_visualization, _infer_chart_type, _build_vega_spec, _compute_aggregates.
- **src/visualization_tool/models.py** — VisualizationRequest/Response, VisualizationOptions.
- **src/markdown_summary_tool/models/schemas.py** — DataRef (shared).
- **src/markdown_summary_tool/engine/duckdb_engine.py** — DuckDBEngine.group_by().

- **Empty question** → ValueError → error_code "visualization_failed".
- **Bad JSON** → caught, returned as error envelope (never raises).
- **All datasets empty** → warning + empty chart_data.
- **Unknown kind** → warning "unknown dataset kind ... skipped".
- **Agg failure** → warning + fallback to raw rows.

*Tip: Use **chart_type="bar"** explicitly when auto inference is ambiguous. For time-series, include "over time" or "trend" in the question to trigger line chart.*